

⇒ Exercise 13: Implement a More Accurate Thrust Model

Rocket thrust is given by...

$$T = \dot{m} v_e + (p_e - p(h)) A_e$$

need...

$$A_e, \dot{m}, v_e, p_e, \underbrace{p(h)}_{f(h)}$$

↑ ↑ ↑
const.

Required User inputs:

$\dot{m}$ = mass flow rate of propellant

v_e = exhaust^{gas} exit velocity

p_e = exhaust^{gas} pressure at exit plane

A_e = nozzle area at exit plane

Calculating Atmospheric Conditions (Temp, Pressure, Air Density, & Speed of Sound) Using C++

by Robert Yager

1. Introduction

This report presents a set of functions, written in C++, that can be used to calculate atmospheric conditions (temperature, pressure, air density, and speed of sound), as well as gravitational-field strength. The functions are based on the atmospheric model presented in *U.S. Standard Atmosphere, 1976*.¹

A summary sheet is provided at the end of this report. It presents the yAtmosphere namespace, which contains the five functions that are described in detail in this report. Also presented is an example that can be used to recreate portions of the “Main Tables” presented in *U.S. Standard Atmosphere, 1976*.

2. Summary of Constants

The following is a list of constants that are used by the code presented in this report. All are from *U.S. Standard Atmosphere, 1976*. Page references refer to the source document.

$R^* = 8314.32 \text{ N m/(kmol K)}$	universal gas constant	(page 3)
$r_0 = 6,356,766 \text{ m}$	effective radius of the earth	(page 4)
$\gamma = 1.400$	specific heat ratio	(page 4)
$T_0 = 288.15 \text{ K}$	standard temperature at sea level	(page 4)
$g'_0 = 9.80665 \text{ m}^2/(\text{s}^2\text{m}')$	geopotential constant	(page 8)
$g_0 = 9.80665 \text{ m/s}^2$	gravitational-field strength at sea level	(page 8)
$\Gamma = 1 \text{ m'/m}$	unit-converting constant	(page 8)
$M_0 = 28.9644 \text{ kg/kmol}$	mean molecular weight of air	(page 9)
$P_0 = 101325.0 \text{ Pa}$	standard pressure at sea level	(page 12)

¹National Aeronautics and Space Administration. *U.S. Standard Atmosphere, 1976*; NASA-TM-X-74335; U.S. Government Printing Office: Washington, DC, October 1976.

3. Summary of Equations

The following is a list of equations that are used by the code presented in this report. All are from *U.S. Standard Atmosphere, 1976*. Equation numbers refer to the source document.

$$g = g_0 \left(\frac{r_0}{r_0 + z} \right)^2 \quad \text{gravitational-field strength} \quad (\text{equation 17})$$

$$H = \frac{Gr_0 z}{r_0 + z} \quad \text{geopotential height} \quad (\text{equation 18})$$

$$T_M = T_{M,b} + L_{M,b}(H - H_b) \quad \text{molecular-scale temperature} \quad (\text{equation 23})$$

$$P = \begin{cases} P_b \left[\frac{T_{M,b}}{T_{M,b} + L_{M,b}(H - H_b)} \right]^{\frac{g'_0 M_0}{R^* L_{M,b}}} & \text{for } L_{M,b} \neq 0 \\ P_b e^{\frac{-g'_0 M_0 (H - H_b)}{R^* T_{M,b}}} & \text{for } L_{M,b} = 0 \end{cases} \quad \text{pressure} \quad (\text{equation 33ab})$$

$$\rho = \frac{PM_0}{R^* T_M} \quad \text{air density} \quad (\text{equation 42})$$

$$c_{\text{sound}} = \left(\frac{\gamma R^* T_M}{M_0} \right)^{1/2} \quad \text{speed of sound} \quad (\text{equation 50})$$

4. Augmented Table 4

Along with equations 23 and 33ab, *U.S. Standard Atmosphere, 1976* specifies the use of tabulated values to calculate temperature and pressure. Values for H_b and $L_{M,b}$ are listed in the source document's table 4. Values for $T_{M,b}$ and P_b can be found by treating equations 23 and 33ab as recursive, with T_0 and P_0 from section 2 used as initial values. In practice, it is computationally efficient to precalculate values for $T_{M,b}$ and P_b . Table 1 lists the values for H_b and $L_{M,b}$ that were originally listed in the source document's table 4, as well as precalculated values for $T_{M,b}$ and P_b .

Table 1. Tabulated values for H_b , $L_{M,b}$, $T_{M,b}$, and P_b .

b	H_b (m')	$L_{M,b}$ (K/m')	$T_{M,b}$ (K)	P_b (Pa)
0	0	-0.0065	288.15	1.01325000000000E+05
1	11000	0	216.65	2.26320639734629E+04
2	20000	0.001	216.65	5.47488866967777E+03
3	32000	0.0028	228.65	8.68018684755228E+02
4	47000	0	270.65	1.10906305554966E+02
5	51000	-0.0028	270.65	6.69388731186873E+01
6	71000	-0.002	214.65	3.95642042804073E+00
7	84852	0	186.946	3.73383589976215E-01

5. Calculating Temperature and Pressure

The following method is valid for $-5,000 \text{ m} \leq z < 86,000 \text{ m}$. Begin by calculating geopotential height (H) as a function of altitude (z) using equation 18. Next, find b | $H_b \leq H < H_{b+1}$. If $H < 0 \text{ m'}$, use $b = 0$. If $H \geq 8,4852 \text{ m'}$, use $b = 7$.

To find molecular-scale temperature, substitute values for H_b , $L_{M,b}$, and $T_{M,b}$ into equation 23.

To find pressure, substitute values for H_b , $L_{M,b}$, $T_{M,b}$, and P_b into equation 33ab.

6. C++ Implementation

6.1 TABLE4 Array

The values for H_b , $L_{M,b}$, $T_{M,b}$, and P_b are stored in the TABLE4 array.

TABLE4 Code

```
const double TABLE4[8][4]={//<=====TRANSITION POINTS
    00000 , -0.0065 , 288.150 , 1.01325000000000E+5 ,//      FOR PRESSURE &
    11000 ,  0.0000 , 216.650 , 2.26320639734629E+4 ,//      TEMPERATURE VS
    20000 ,  0.0010 , 216.650 , 5.47488866967777E+3 ,//      GEOPOTENTIAL
    32000 ,  0.0028 , 228.650 , 8.68018684755228E+2 ,//      ALTITUDE CURVES
    47000 ,  0.0000 , 270.650 , 1.10906305554966E+2 ,//      [table 4]
    51000 , -0.0028 , 270.650 , 6.69388731186873E+1 ,//      (3RD COLUMN IS
    71000 , -0.0020 , 214.650 , 3.95642042804073E+0 ,//      TEMPERATURE,
    84852 ,  0.0000 , 186.946 , 3.73383589976215E-1 ,//      4TH, PRESSURE)
};//~~~YAGENAUT@GMAIL.COM~~~~~LAST~UPDATED~15MAR2013~~~~~
```

6.2 Temperature() Function

The Temperature() function uses the method described in section 5 to calculate molecular-scale temperature as a function of altitude. For altitudes up to 80,000 m, molecular-scale temperature is equal to absolute temperature. At 80,000 m, molecular-scale temperatures begin to differ from absolute temperatures. At 86,000 m (the limit of this model), *U.S. Standard Atmosphere, 1976* states that the difference between molecular-scale temperature and absolute temperature is 0.0787 K.

Temperature() Code

```
inline double Temperature(//<=====TEMPERATURE (K)
    double z){//<-----ALTITUDE (m) (T IS VALID FOR -5,000 m < z < 86,000 m)
    double H=z*6356766/(z+6356766);//.....[equation 18]
    int b; /*<*/for(b=0;b<7;++b)if(H<TABLE4[b+1][0])break;
    return TABLE4[b][2]+TABLE4[b][1]*(H-TABLE4[b][0]);//.....[equation 23]
} //~~~~~YAGENAUT@GMAIL.COM~~~~~LAST~UPDATED~15MAR2013~~~~~
```

Temperature() Parameters

z **z** is a height above sea level (in meters). Output of the Temperature() function is valid for $-5000 \text{ m} \leq z \leq 86,000 \text{ m}$. Output for $z = -6356766 \text{ m}$ (i.e., the center of the earth) is undefined.

Temperature() Return Value

The return value of the Temperature() function is the molecular-scale temperature, in Kelvin, at the given altitude (**z**). See the beginning of this section for a note on the minor difference between molecular-scale temperature and absolute temperature.

6.3 Pressure() Function

The Pressure() function uses the method described in section 5 to calculate atmospheric pressure as a function of altitude.

Pressure() Code

```
inline double Pressure(//<=====PRESSURE (Pa)
    double z){//<-----ALTITUDE (m) (P IS VALID FOR -5,000 m < z < 86,000 m)
    double H=z*6356766/(z+6356766);//.....[equation 18]
    int b; /*<*/for(b=0;b<7;++b)if(H<TABLE4[b+1][0])break;
    double C=-.0341631947363104; //.....C = -G0*M0/RSTAR [pages 8,9,3]
    double Hb=TABLE4[b][0], Lb=TABLE4[b][1], Tb=TABLE4[b][2], Pb=TABLE4[b][3];
    return Pb*(fabs(Lb)>1E-12?pow(1+Lb/Tb*(H-Hb), C/Lb):exp(C*(H-Hb)/Tb));
} //~~~~~YAGENAUT@GMAIL.COM~~~~~LAST~UPDATED~15MAR2013~~~~~
```


Pressure() Parameters

z **z** is a height above sea level (in meters). Output of the Pressure() function is valid for $-5000 \text{ m} \leq z \leq 86,000 \text{ m}$. Output for $z = -6356766 \text{ m}$ (i.e., the center of the earth) is undefined.

Pressure() Return Value

The return value of the Pressure() function is the pressure, in Pascals, at the given altitude (**z**).

6.4 Density() Function

The Density() function uses equation 42 to calculate air density as a function of temperature and pressure.

Density() Code

```
inline double Density(//<=====DENSITY (kg/m^3)
    double T, //<-----TEMPERATURE (K) (CALCULATE T USING Temperature())
    double P){ //<-----PRESSURE (Pa) (CALCULATE P USING Pressure())
    return P*.00348367635597379/T; //.....[equation 42]
} //~~~~YAGENAUT@GMAIL.COM~~~~~LAST~UPDATED~15MAR2013~~~~~
```

Density() Parameters

T **T** is a local temperature (in Kelvin). Values for **T** can be determined using the Temperature() function. Output for **T** = 0 K is undefined.

P **P** is a local pressure (in Pascals). Values for **P** can be determined using the Pressure() function.

Density() Return Value

The return value of the Density() function is the local density of air, in kilograms/cubic meter.

6.5 SpeedofSound() Function

The SpeedofSound() function uses equation 50 to calculate speed of sound as a function of temperature.

SpeedofSound() Code

```
inline double SpeedofSound(//<=====SPEED OF SOUND (m/s)
    double T){ //<-----TEMPERATURE (K) (CALCULATE T USING Temperature())
    return sqrt(401.87430086589*T); //.....[equation 50]
} //~~~~YAGENAUT@GMAIL.COM~~~~~LAST~UPDATED~15MAR2013~~~~~
```

SpeedofSound() Parameters

T **T** is a local temperature (in Kelvin). Values for **T** can be determined using the Temperature() function.

SpeedofSound() Return Value

The return value of the SpeedofSound() function is the local speed of sound, in meters per second.

6.6 Gravity() Function

The Gravity() function uses equation 17 to calculate the gravitational-field strength near the surface of the earth as a function of altitude.

Gravity() Code

```
inline double Gravity(//<=====ACCELERATION DUE TO GRAVITY (m/s^2)
    double z){//<-----ALTITUDE (m) (g IS VALID FOR -5,000 m < z < 86,000 m)
    return 9.80665*pow(1+z/6356766,-2);//.....[equation 17]
} //~~~~YAGENAUT@GMAIL.COM~~~~~LAST~UPDATED~15MAR2013~~~~~
```

Gravity() Parameters

z **z** is a height above sea level (in meters). The output of the Gravity() function is valid for $-5000 \text{ m} \leq z \leq 1,000,000 \text{ m}$. Output for $z \leq -6356766 \text{ m}$ is undefined.

Gravity() Return Value

The return value of the Gravity() function is the gravitational-field strength, in meters per second squared.

7. Summary

A summary sheet is provided at the end of this report. It presents the yAtmosphere namespace, which contains the five functions described in detail in section 6. Also presented is an example that can be used to recreate portions of the “Main Tables” presented in *U.S. Standard Atmosphere, 1976*.¹

Some values that are calculated using the example code differ from values presented in *U.S. Standard Atmosphere, 1976* in the least-significant figure. For example, at an altitude of 85,500 m, *U.S. Standard Atmosphere, 1976* states that the molecular-scale temperature will be 187.920 K, while the example code predicts that the molecular-scale temperature will be 187.919 K.